HTR Voice System — Complete Documentation

Version: 1.0
Date: May 2026
Status: Production (Chrome / Edge only)

Table of Contents

---

1. User Guide

What is the Voice System?

The HTR platform has a built-in voice interface that lets you navigate the entire application, search for content, ask the AI Analyst questions, and control the interface — all hands-free using your microphone.

It works in two directions:

- **Voice Input:** You speak → the app acts (navigate, search, fill in text)
- **Voice Output:** The AI speaks responses back to you when voice mode is on

---

How to Turn Voice On and Off

Method 1 — The Mic Button

There is a round dark button fixed at the bottom-center of every page. It always shows a microphone icon.

- **Click it once** → voice turns ON (button turns red and pulses)
- **Click it again** → voice turns OFF (button returns to dark/grey)

Method 2 — Keyboard Shortcut

Press ⌘ **Shift V** (Mac) or **Ctrl Shift V** (Windows/Linux) at any time from any page.

Same toggle: first press turns voice on, second press turns it off.

Visual States of the Mic Button

Button appearance	What it means
Dark grey, mic icon, label "Voice off — click to start (⌘⇧V)"	Voice is OFF
Red, pulsing ring, mic icon, label "Listening… click to stop"	Voice is ON and listening
Indigo/blue, pulsing ring, speaker icon, label "Speaking — click to stop"	AI is reading a response aloud
Label "Checking mic support…"	Page just loaded, detecting browser support

There is also a small mic icon inside the search bar in the header. It turns red when voice is active.

First-Time Setup: Granting Microphone Permission

The first time you turn voice on, your browser will show a popup asking:

> **"localhost wants to use your microphone"**

Click **Allow**. The browser will remember this choice. If you accidentally clicked Block, see the Troubleshooting section below.

What You Can Do With Your Voice

There are three modes of voice input, chosen automatically based on what you say:

Mode 1 — Navigation Commands (executes immediately, no confirmation needed)

Say any of these and the app navigates instantly:

- "*Go to policy*" → opens the Policy section
- "*Go to economics*" → opens Economics
- "*Open the AI Analyst*" → opens the right-side chat panel
- "*Go home*" → goes to the homepage

See the full navigation command list in Section 2.

Mode 2 — Search (fills the search box, you press Enter to search)

Say anything that doesn't match a command and you're not on the chat page:

- "*What is value-based care*" → fills the search box with that phrase
- "*Medicaid work requirements*" → fills search with those words

The search box is focused and ready — press Enter or click Search to run it.

Mode 3 — AI Chat (fills the chat textarea, you press Enter to send)

When you're on the `/chat` page or have the AI Analyst sidebar open:

- "*What are the 2026 Medicaid income limits in Vermont?*" → fills the chat input
- "*Explain AHEAD model*" → fills the chat input

Press Enter to send, or just read it over before sending.

Voice Output — AI Reads Responses Aloud

When voice mode is ON and the AI Analyst responds to a question, the response is automatically read aloud after the full response has streamed in.

- The mic button turns indigo/blue with a speaker icon while speaking
- A "Speaking…" label appears above the button
- **To stop the AI mid-sentence:** click the button once, or say **"stop"**

Voice output only happens when voice mode is active. If you turn voice off, the AI will not speak.

Troubleshooting

Nothing happens when I click the mic button

- Make sure you are using Chrome or Edge — Firefox and Safari do not support voice input
- Check that you granted microphone permission (see below)

I blocked microphone access by mistake

- In Chrome: click the padlock icon in the address bar → Site settings → Microphone → Allow
- Or go to `chrome://settings/content/microphone`, find localhost, and change to Allow
- Reload the page after changing the setting

The AI is reading responses but I don't want that

- Turn voice mode off (click the mic button or press ⌘⇧V) — voice output only happens when voice mode is on

Voice heard the wrong word / navigation went to the wrong page

- The system uses fuzzy matching — if it hears a partial word that matches a page title, it will navigate there
- Speak clearly and use the exact command phrases listed in Section 2
- Acronyms like "HIE" should be spelled out: say "health information exchange" instead

---

2. Full Voice Command Reference

Controls

Say this	What happens
(click mic button) or ⌘⇧V	Toggle voice on/off
`"stop"`	Stops the AI from speaking mid-response
`"cancel"`	Same as "stop"
`"never mind"`	Same as "stop"

Navigation — Main Sections

Say this	Goes to
`"go to home"` or `"home"`	`/` (homepage)

Say this	Goes to
`"go to economics"` or `"economics monitor"`	`/economics`
`"go to policy"` or `"policy analysis"`	`/policy`
`"go to technology"` or `"technology radar"`	`/technology`
`"go to clinical"` or `"clinical intelligence"`	`/clinical`
`"go to health equity"` or `"equity"`	`/equity`

Note: You can also say the section name by itself (e.g. "*economics*") — the system will match it against the full command list.

Navigation — Dashboards & Tools

Say this	Goes to
`"national dashboard"`	`/dashboard`
`"investment tracker"`	`/economics/investment`
`"HTI simulator"`	`/hti-dashboard`
`"go to AI analyst"`	`/chat` (full chat page)
`"go to research lab"`	`/research-lab`
`"go to state dashboard"`	`/dashboard`
`"go to community"`	`/community`
`"go to account"`	`/account`
`"go to pricing"`	`/pricing`

Navigation — State Profiles

Say this	Goes to
`"Vermont profile"`	`/dashboard/vermont`
`"Texas profile"`	`/dashboard/texas`
`"California profile"`	`/dashboard/california`
`"New York profile"`	`/dashboard/new_york`

Navigation — Actions

Say this	What happens
`"start new conversation"`	Opens `/chat`
`"upgrade plan"`	Opens `/pricing`
`"view saved articles"`	Opens `/account/bookmarks`
`"take annual survey"`	Opens `/survey`

Navigation — Research Lab Tools

Say this	Goes to
`"open APM calculator"`	`/research-lab?tool=apm`
`"open CEA calculator"`	`/research-lab?tool=cea`
`"open HCC scoring"`	`/research-lab?tool=hcc`
`"open FHIR lab"`	`/research-lab?tool=fhir`

Interface Controls

Say this	What happens
`"open sidebar"` or `"show sidebar"`	Opens the left navigation sidebar
`"close sidebar"` or `"hide sidebar"`	Closes the left navigation sidebar
`"open AI"` or `"open analyst"` or `"show AI"`	Opens the right AI Analyst sidebar
`"close AI"` or `"close analyst"`	Closes the right AI Analyst sidebar
`"command palette"` or `"open palette"`	Opens the ⌘K command palette

Text Input (anything not matching a command above)

Context	What happens
On any page except `/chat`	Text fills the header search box; press Enter to search
On the `/chat` page	Text fills the chat textarea; press Enter to send
AI Analyst sidebar is open (non-chat page)	Text fills the sidebar textarea; press Enter to send

---

3. Tips for Best Recognition

Speak naturally but clearly
The Web Speech API uses Google's speech recognition engine. Speak at a normal pace — don't slow down artificially.

Pause before and after commands
Start speaking after the button turns red. The system processes one utterance at a time and restarts automatically between phrases.

Use exact page names for navigation
The system matches what you say against the exact titles in the command list. "Go to economics" works better than "take me to the economics section please."

Spell out acronyms
Say "health information exchange" instead of "HIE." The speech engine may transcribe "HIE" as "h i e" letter by letter.

Quiet environment
Background noise can confuse the speech engine. The mic captures everything in the room.

Voice mode stays on across page navigation
If you navigate to a new page while voice is on, voice stays on. You don't need to re-enable it.

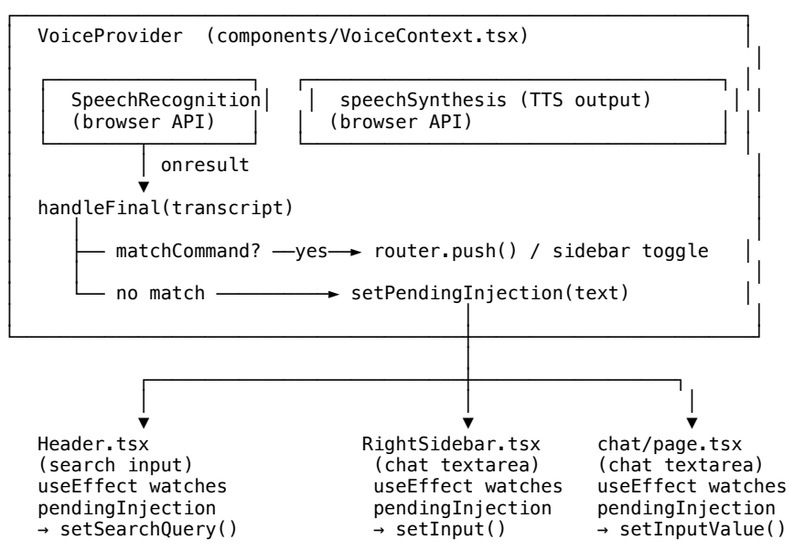
AI responses are read in full
The AI response is only spoken after the complete response has finished streaming — not word by word. For long responses this means a few seconds of silence before speaking begins.

---

4. Technical Architecture

Overview

The voice system is built entirely on browser-native Web APIs — no third-party libraries, no backend calls for speech processing, no cost per use.



The "ref-bag" Pattern

The core engineering challenge with continuous voice recognition in React is **stale closures**. The `SpeechRecognition` callbacks (`onresult`, `onend`, `onerror`) are registered once when `rec.start()` is called. If they capture `router`, `sidebar`, or any state directly, those values go stale as React re-renders.

The solution is a single mutable ref object (`bag`) that holds all values the callbacks need:

```
const bag = useRef({
  router,
  sidebar,
  pathname,
  wantListening: false,
  recognition: null as SpeechRecognition | null,
  setIsListening,
  setTranscript,
  setIsSpeaking,
  setPendingInjection,
});
```

Three `useEffect` hooks keep `bag.current.router`, `bag.current.sidebar`, and `bag.current.pathname` in sync whenever React re-renders. Recognition callbacks always read from `bag.current.*` — they never capture any closure variable directly.

`wantListening` is also stored in the bag (not as React state) because the `onend` callback needs to check it synchronously to decide whether to restart recognition, and reading React state inside a callback sees the stale value from when the callback was registered.

Recognition Lifecycle

```
toggleListening()
   │
   ├── wantListening was false
   │     → set wantListening = true
   │     → setIsListening(true)   [React state → UI update]
   │     → startRecognition()
   │           → new SpeechRecognition()
   │           → rec.start()
   │           → onstart fires    [logs "[Voice] started"]
   │           → onresult fires per utterance
   │                 → interim results → setTranscript() → live pill UI
   │                 → final result   → handleFinal(text)
   │           → onend fires
   │                 → if wantListening: setTimeout(startRecognition, 150ms)
   │                 → restart loop (continuous listening)
   │
   └── wantListening was true
         → set wantListening = false
         → setIsListening(false)  [React state → UI update]
         → rec.abort()
         → recognition = null
```

The 150ms delay on restart prevents hammering the API when `onend` fires immediately (e.g. network hiccup, no-speech timeout).

Command Routing Logic

`handleFinal(rawText)` processes every final transcript in this priority order:

1. **Stop commands** — `"stop"`, `"cancel"`, `"never mind"` → calls `stopSpeaking()`
2. **Command palette** — `"command palette"` → dispatches synthetic Cmd+K KeyboardEvent
3. **Sidebar controls** — open/close left or right sidebar
4. **Direct title match** — scans every entry in `COMMANDS[]`, checks if the lowercase transcript includes the lowercase command title
5. **Prefix match** — if transcript starts with `"go to"`, `"navigate to"`, `"take me to"`, `"show me"`, or `"open"`, extracts the destination and fuzzy-matches against COMMANDS titles
6. **Text injection fallback** — if nothing matched, sets `pendingInjection` state

---

Text Injection Flow

`pendingInjection` is a `string | null` in VoiceContext. When it's set, three `useEffect` hooks in three different components race to consume it:

- **Header.tsx** — watches `pendingInjection`, skips if `pathname === "/chat"`, otherwise sets `searchQuery` state and focuses `searchInputRef`
- **RightSidebar.tsx** — watches `pendingInjection`, skips if `pathname === "/chat"`, otherwise sets `input` state and focuses `textareaRef`
- **chat/page.tsx** — watches `pendingInjection`, always consumes it (no pathname guard needed — this component only mounts on `/chat`), sets `inputValue` and resizes the textarea

The consuming component calls `voice.clearInjection()` immediately after reading, which sets `pendingInjection` back to `null`. The other two watchers see `null` and do nothing.

TTS Output

After each completed AI stream, the relevant component calls `voice.speakText(finalText)`.

`speakText()`:

1. Returns immediately if `wantListening` is false (voice mode is off)
2. Calls `stripMarkdown(text)` to remove `**`, `*`, `#`, backticks, citation sentinels, and markdown links before reading
3. Cancels any currently-playing utterance
4. Creates a new `SpeechSynthesisUtterance` at rate 0.92 (slightly slower than default)
5. Sets `isSpeaking = true` → mic button UI switches to speaker icon
6. On `utterance.onend` / `utterance.onerror` → sets `isSpeaking = false` → UI reverts

Two components call `speakText()`:

- `RightSidebar.tsx` line 219 — after the stream finishes and final citations are parsed
- `chat/page.tsx` line ~460 — after the stream finishes and final citations are parsed

---

SSR Safety

The voice system runs entirely client-side. Three measures prevent Next.js server-side rendering conflicts:

1. **`isSupported` starts as `false`** — set only inside `useEffect`, which never runs on the server. Both server and client render the same initial `false`, eliminating hydration mismatch.

2. **VoiceFab is dynamically imported with `ssr: false`** — it is never rendered on the server at all.

3. **Mic button in Header is gated on `mounted` state** — the button only appears after the Header component's own `useEffect(() => setMounted(true))` fires, guaranteeing the server and initial client renders match exactly.

---

5. File Map

File	Role
`frontend/components/VoiceContext.tsx`	Global context — owns all voice state, recognition lifecycle, command routing, TTS
`frontend/components/VoiceFab.tsx`	Floating mic/speaker button rendered on every page
`frontend/components/ClientOnlyShell.tsx`	Dynamically imports VoiceFab (and CommandPalette, SessionTimeout) with `ssr: false`
`frontend/app/layout.tsx`	Wraps the app in `<VoiceProvider>` inside `<SidebarProvider>`
`frontend/components/Header.tsx`	Mic icon in search bar; `useEffect` consumes `pendingInjection` → search field
`frontend/components/RightSidebar.tsx`	`useEffect` consumes `pendingInjection` → sidebar textarea; calls `speakText()` after AI stream
`frontend/app/chat/page.tsx`	`useEffect` consumes `pendingInjection` → chat textarea; calls `speakText()` after AI stream
`frontend/components/CommandPalette.tsx`	Exports `COMMANDS[]` array — imported by VoiceContext for navigation matching
`frontend/next.config.ts`	`Permissions-Policy: microphone=(self)` — allows mic access from own origin only

---

6. API & Context Reference

`useVoice()` hook

Import: `import { useVoice } from "@/components/VoiceContext";`

Returns the `VoiceContextType` object:

```
interface VoiceContextType {
  // State (read-only from consumers)
  isListening: boolean;       // true when mic is active and recording
  isSpeaking: boolean;        // true when TTS is playing
  isSupported: boolean;       // true after mount if browser supports SpeechRecognition
  transcript: string;         // live interim text while user is speaking (resets on final)
  pendingInjection: string | null;  // non-null when a non-command transcript is ready

  // Actions
  toggleListening: () => void;      // start or stop the microphone
  clearInjection: () => void;       // mark pendingInjection as consumed (set to null)
  speakText: (text: string) => void; // read text aloud via TTS (only if voice is on)
  stopSpeaking: () => void;         // cancel any active TTS utterance
}
```

`VoiceProvider`

Import: `import { VoiceProvider } from "@/components/VoiceContext";`

Wrap any subtree that needs voice access. In this app it wraps everything inside `<SidebarProvider>` in `layout.tsx` so it has access to `useSidebar()` for sidebar toggle commands.

COMMANDS array

Import: `import { COMMANDS } from "@/components/CommandPalette";`

Used internally by VoiceContext. Each entry:

```
type CommandItem = {
  id: string;
  title: string;         // matched against voice transcript
  category: "Navigation" | "State" | "Tool" | "Actions" | "Launch Tool";
  href: string;          // route pushed on match
  icon: React.ComponentType;
  shortcut?: string;     // keyboard shortcut letter (not used by voice)
}
```

To add new navigation destinations reachable by voice, add entries to the `COMMANDS` array in `CommandPalette.tsx`. No changes to VoiceContext are needed.

---

7. Browser Compatibility & Security

Browser Support

Browser	Voice Input	Voice Output
Chrome (desktop)	✅ Full support	✅ Full support
Edge (desktop)	✅ Full support	✅ Full support
Safari (macOS/iOS)	❌ Not supported	✅ Works
Firefox	❌ Not supported	✅ Works
Chrome (Android)	✅ Full support	✅ Full support

When `isSupported` is false, the mic button still renders but shows "Checking mic support…" — it does nothing on click. No crash or error is shown to the user.

Security

Microphone permission: The browser asks once. Permission is stored per origin (localhost or the production domain). The server's `Permissions-Policy: microphone=(self)` header ensures no embedded iframe or third-party script can request microphone access.

All processing is local: Speech-to-text is processed by Google's servers via the browser's built-in `SpeechRecognition` API (same as Chrome's address bar voice search). No audio is sent to HTR's own servers. No audio is stored.

Voice output uses browser TTS: `speechSynthesis` is entirely local — the browser's built-in text-to-speech engine. No audio leaves the device.

PHI warning: The existing PHI detection in `chat/page.tsx` applies to voice-injected text the same as typed text — if the transcript contains patterns matching SSN, MRN, or name+DOB combinations, the send will be blocked and a warning shown.

Production Deployment

The `Permissions-Policy: microphone=(self)` header is already set in `next.config.ts` and applies to all environments. No additional configuration is needed for production.

---

8. Known Limitations

One utterance at a time
`SpeechRecognition` is configured with `continuous: false`. It listens for one utterance, processes it, then automatically restarts. There is a ~150ms gap between utterances. You cannot speak a very long continuous sentence — pause naturally between phrases.

No wake word
There is no always-listening "Hey HTR" wake word. Voice must be manually toggled on with the button or ⌘⇧V. This is intentional — always-on listening raises privacy concerns and drains battery.

Acronyms are transcribed literally
"HIE" may be heard as "h i e", "EHR" as "e h r", etc. Speak the full phrase ("health information exchange", "electronic health record") for reliable recognition.

Navigation matching is greedy
The command router scans COMMANDS in order and matches on the first title found in the transcript. If your search query happens to contain a page name (e.g., "what is the Vermont Profile dashboard") it may navigate instead of searching. Use more specific language if this occurs.

TTS reads the full response, not streaming
Voice output waits for the complete AI response before speaking. For long responses (>500 words) there will be a noticeable delay before speech begins.

Voice state does not persist across hard reloads
If you reload the page, voice is off by default. There is no auto-start option.

Tab/window isolation
Voice is per browser tab. Opening the app in two tabs means each tab has its own independent voice state.

---

9. How to Extend

Adding new navigation voice commands

Edit `frontend/components/CommandPalette.tsx`, add to the `COMMANDS` array:

```
{ id: "nav-99", title: "My New Page", category: "Navigation", href: "/my-new-page", icon: SomeIcon },
```

The voice system will immediately recognize phrases like:

- "*go to my new page*"
- "*my new page*"
- "*open my new page*"

No changes to `VoiceContext.tsx` needed.

Adding new control commands

Edit the `handleFinal()` function in `frontend/components/VoiceContext.tsx`. Add a new `if` block before the COMMANDS loop:

```
if (t.includes("dark mode")) {
  bag.current.setTheme("dark");
  return;
}
```

Adding voice injection to a new text field

In the target component:

```
import { useVoice } from "@/components/VoiceContext";

const voice = useVoice();

useEffect(() => {
  if (!voice.pendingInjection) return;
  // Add any condition to decide if THIS component should consume it
  // e.g. only consume when this component is visible/focused
  setMyInputState(voice.pendingInjection);
  voice.clearInjection();
  myInputRef.current?.focus();
}, [voice.pendingInjection, voice]);
```

Important: Call `voice.clearInjection()` immediately after reading the value. Only one component should consume each injection — the first one that matches the condition wins.

Changing the TTS voice or speed

In `frontend/components/VoiceContext.tsx`, in the `speakText()` function:

```
const utt = new SpeechSynthesisUtterance(stripMarkdown(text));
utt.rate = 0.92;       // 0.1 (very slow) to 10 (very fast), default 1
utt.pitch = 1;         // 0 to 2, default 1
utt.volume = 1;        // 0 to 1, default 1

// To use a specific voice:
const voices = window.speechSynthesis.getVoices();
utt.voice = voices.find(v => v.name === "Samantha") ?? null; // macOS voice
```

Replacing Web Speech API with OpenAI Whisper

If you need Firefox/Safari support or higher accuracy, the architecture supports swapping the recognition backend. In `startRecognition()` in `VoiceContext.tsx`, replace the `SpeechRecognition` block with:

1. `navigator.mediaDevices.getUserMedia({ audio: true })` to get the mic stream
2. `MediaRecorder` to capture audio chunks
3. POST the audio blob to `/api/voice` (a new Next.js API route)
4. The API route forwards to OpenAI Whisper (`whisper-1` model) and returns the transcript
5. Call `handleFinal(transcript)` with the returned text

The rest of the system (command routing, injection, TTS) requires no changes.